

Addressing Recurring Concept Drift in Data Streams Proactively

Project Team

Muhammad Omer Nasir 21I-2476
Muhammad Yahya 21I-2592

Session 2021-2025

Supervised by

Ms. Tajwar Mehmood



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Domain	1
1.2	Research Problem Statement	2
1.3	Software Requirements Specification	3
1.3.1	Functional Requirements	3
1.3.2	Non-Functional Requirements	4
2	Literature Review	7
2.1	Related Research	7
2.1.1	A lightweight concept drift detection and adaptation framework for IoT data streams.	7
2.1.1.1	Summary of the research item	7
2.1.1.2	Critical analysis of the research item	8
2.1.1.3	Relationship to the proposed research work	9
2.1.2	Disposition-Based Concept Drift Detection and Adaptation in Data Stream	10
2.1.2.1	Summary of the research item	10
2.1.2.2	Critical analysis of the research item	10
2.1.2.3	Relationship to the proposed research work	11
2.1.3	Learning in the Presence of Concept Recurrence in Data Stream Clustering	12
2.1.3.1	Summary of the research item	12
2.1.3.2	Critical analysis of the research item	12
2.1.3.3	Relationship to the proposed research work	14
2.1.4	Nacre: Proactive Recurrent Concept Drift Detection in Data Streams	14
2.1.4.1	Summary of the research item	14
2.1.4.2	Critical analysis of the research item	18
2.1.4.3	Relationship to the proposed research work	18
2.1.5	Recurrent Adaptive Classifier Ensemble for Handling Recurring Concept Drifts	19
2.1.5.1	Summary of the research item	19
2.1.5.2	Critical analysis of the research item	20

2.1.5.3	Relationship to the proposed research work	21
3	Proposed Approach	23
3.1	Workflow of the Approach	23
3.2	Key features of the proposed approach	24
3.3	Visualization of the Dashboard	27
3.3.1	Purpose of the Dashboard	27

List of Figures

2.1	The framework of the proposed drift adaptive IoT data analytics model . .	8
2.2	Proposed Learning System	13
2.3	NACRE Framework Overview	15
2.4	Synthetic Dataset Performance	16
2.5	Real-world Dataset Performance	17
2.6	RACE Memory Consumption	20
2.7	RACE CPU Processing Time	21
3.1	Proposed workflow for DriftGuard	23
3.2	Monthly Time-Series analysis for Sales and shipping	28

List of Tables

Chapter 1

Introduction

Since data science is in evolution, it's crucial that we can correctly interpret changes in data patterns and behave accordingly. This chapter introduces the problem domain through the deep study of this problem domain and articulates the central research problem. This chapter, by first establishing the theoretical foundation and motivation, establishes the problem of real-time concept drift detection within streaming data environments, and from there dives into the knurled details. It covers the gaps in current knowledge, the problems raised by shifting dynamic data, and the technical and theoretical objectives of this study.

The goal of the study is to provide a novel framework for continuous detection of, and response to, recurring concept drift, a problem that existing methods largely disregard in the event of either abrupt or gradual drift, absent hitherto from consideration of recurrent changes in data patterns in some domains. In addition, as a prelude for the technical requirements and software architecture that will help support the research, in cases where relevant, this introduction provides the reader holistic understanding of the scope and direction of the study.

1.1 Problem Domain

Real-time data analytics continues to become more common involving large volumes of data, and the necessity for producing usable predictions in near real-time increases as a result, which has led in turn to the increasing importance of the concept drift problem — the phenomenon that the statistical properties of target variables in such models change over time. This research is grounded in the problem of addressing this phenomenon, especially in domains where data is continuously generated, namely e-commerce, supply chain management, and automated decision systems. In such environments, stationarity of data distributions is an unrealistic assumption resulting in model performance degrading

over time as patterns change.

Despite the wealth of literature on concept drift, existing approaches typically focus on handling abrupt or gradual drifts adhering to the intuition that shifts in data distribution occur abruptly or evolve gradually over time. In a few domains including supply chain, the drifting of recurring concepts is a unique challenge. Recurring drift occurs when old properties, patterns, or distributions are seen again and again, usually caused by cyclic or seasonal things like seasonally changing consumer behavior around holiday sales or predictable market conditions. But these drifts are not random; they are part of identifiable cycles and so a detection mechanism must be adaptive, nuanced, and tuned to their time scale.

However, these recurring drifts tend to be inadequately addressed by current detection methodologies. Most approaches are reactive and discover drifts only after their occurrence, or they are proactive and can predict drifts but often at the computational and scalability cost. The research problem, therefore, lies at the intersection of these challenges: a real-time detection system can work with large-scale data streams, and yet be able to account for repeating patterns that the traditional methods could miss out.

The consequences of failing to detect recurring drift are profound. Real-time decision-making means that the models cannot adapt to such changes and can end up making outdated or wrong predictions which can result in large monetary losses in business settings. An example is supply chain logistics where supply and demand seasonality is disregarded leading to increased or decreased sales and supply line efficiency. The problem addressed in this study sits within a must-refined urgent need for scalable, domain-specific solutions to the complexity of concept drift with recurrence.

1.2 Research Problem Statement

At the core of this research is a fundamental question: Real-time detection and adaptation to recurring concept drift in the context of streaming data streams? It notes that the existing methods do not address the cyclical nature of many data patterns and this is a significant gap in the current body of knowledge.

When we consider the growing reliance on streaming data in modern applications, the problem becomes especially acute. With real-time collection of data, the ability to adapt to changing data distributions is a key factor in keeping machine learning models accurate and relevant. Statistical tests, drift detectors, or adaptive learning techniques that ensure purity or speed tend to sacrifice either one of them in their large-scale, real-time systems. In addition, most current methods are not designed to capture such recurrent drift and therefore do not capture the nuances of periodically repeating patterns that are commonly present in many practical situations.

Thus, the research aims to fill this gap by developing a robust tool capable of identifying recurring drift proactively and adapting in real-time without compromising on computational efficiency. On one hand, this is not an academic challenge, but a practical imperative in industries as diverse as supply chain, where the ability to know and adapt to cycles in sales trends determines a competitive advantage.

The objectives of this study are varied: We first develop an algorithm that can detect recurring drifts in streaming data in real-time and secondly ensure that the algorithm scales appropriately on the high velocity and volume of data present in modern streaming applications, and thirdly demonstrate the practical viability of the approach in a real-world domain where recurring drifts are both predictable and impactful.

1.3 Software Requirements Specification

1.3.1 Functional Requirements

1. Real-Time Drift Visualization:

- The dashboard should display real-time drift events based on the selected feature.
- Provide graphical representations (e.g., line graphs, bar charts) to show the occurrence of concept drift over time.

2. Feature Selection:

- The user can select one or more features from a dropdown to monitor drift patterns.
- The selected features will update the visualizations accordingly to reflect drift changes specific to those features.

3. Historical Drift Data:

- The dashboard should allow users to view historical drift data for comparison with the current stream.
- Users can toggle between real-time and historical views.

4. Drift Prediction Alerts:

- The dashboard should notify the user when a drift is predicted to occur, based on the sequence predictor.
- Alerts should include visual and textual notifications.

5. Drift Details & Concept History:

- The dashboard should display detailed information on current and previous concepts stored in the system.
- Users should be able to see metadata related to each drift occurrence, such as date, time, affected feature, and drift type.

6. Customizable Time Windows:

- The user should have the ability to adjust the time window for which drift data is being monitored (e.g., last hour, last day, etc.).

7. Export Drift Reports:

- The dashboard should allow users to export reports on drift patterns, predictions, and adaptations in CSV or PDF format.

8. Interactive Graphs and Tooltips:

- Users should be able to hover over graphs to see detailed information about each drift point (e.g., confidence, timestamp).

1.3.2 Non-Functional Requirements

1. Performance:

- The dashboard must process and display drift data with minimal latency (updates within 2 seconds).
- It should handle high-volume data streams efficiently without performance degradation.

2. Scalability:

- The dashboard should scale with increasing numbers of features and drift events. (Additional Functionality)
- It should accommodate multiple users monitoring different features simultaneously. (Additional Functionality)

3. Usability:

- The user interface should be intuitive and easy to navigate for users with varying technical expertise.
- Feature selection dropdown should provide search and filter functionality for a number of features.

4. Reliability:

- The dashboard must ensure continuous uptime with fallback mechanisms during data stream interruptions.
- It should maintain accurate drift data during temporary disruptions.

5. Security:

- The dashboard should implement authentication to ensure only authorized users access the data.
- Drift reports and data streams must be encrypted and transmitted securely. (Additional Functionality)

6. Compatibility:

- The dashboard should be accessible on major web browsers (Chrome, Firefox, Safari, Edge) and responsive to various screen sizes. (Additional Functionality)

7. Data Storage:

- The system must optimize data storage, ensuring relevant drift patterns and historical data are retained while minimizing memory usage.

Chapter 2

Literature Review

2.1 Related Research

2.1.1 A lightweight concept drift detection and adaptation framework for IoT data streams.

2.1.1.1 Summary of the research item

This paper (1) proposes a machine learning framework that is both lightweight and can adapt to the issues of concept drift in IoT data streams. Since IoT devices are rapidly growing and are generating huge amounts of data, the characteristics of these streams often result in concept drift especially when the distribution of data changes because of changes in environmental factors or devices, device updates, or abnormal system events. The proposed framework has been established around anomaly detection from IoT data streams using the LightGBM model which has been optimized; along with a new drift, detection, and adaptation called OASW.

The management of these ideas requires a conceptual framework and the conceptual framework comprises two main parts. Initially, a LightGBM machine learning algorithm is applied since it can handle big and high dimensional IoT data better. Hyperparameter tuning is done using Particle Swarm Optimization (PSO) for LightGBM to enhance the performance of the model. The second component that we propose is OASW, a new approach to concept drift adaptation that uses sliding windows for detecting drifts and adaptive windows for updating the model. OASW is capable of dealing with different types of concept drift (incremental, abrupt, cyclical) due to the use of dynamic window size and update of the model whenever there is the need.

Experiments were conducted on two public IoT datasets: Specifically, performance is evaluated on IoTID20 and NSL-KDD datasets which are popular in anomaly detection

and cybersecurity investigation. As it has been shown previously, the presented framework was efficient, providing high accuracy values of 99.92% of IoTID20 dataset and 98.31% of NSL-KDD while having relatively low computational complexity of the proposed algorithm. This makes it particularly suitable for resource-limited IoT applications, which are today's applied in more and more developing countries.

2.1.1.2 Critical analysis of the research item

The paper aims to propose a reliable solution for detecting concept drift in IoT data streams since IoT systems are continuously evolving. Another rather important contribution is the application of LightGBM 2.1 – gradient boosting that is known to work best for large data sets. That is why LightGBM can be effectively used in IoT applications since the devices connected to this network often have limited resources, such as memory or computational power. Furthermore, applying PSO for hyperparameter optimization presents a novel approach of increasing the performance of a model we do not have to spend so much time tuning these parameters since it is both time-wasteful and prodigal of error.

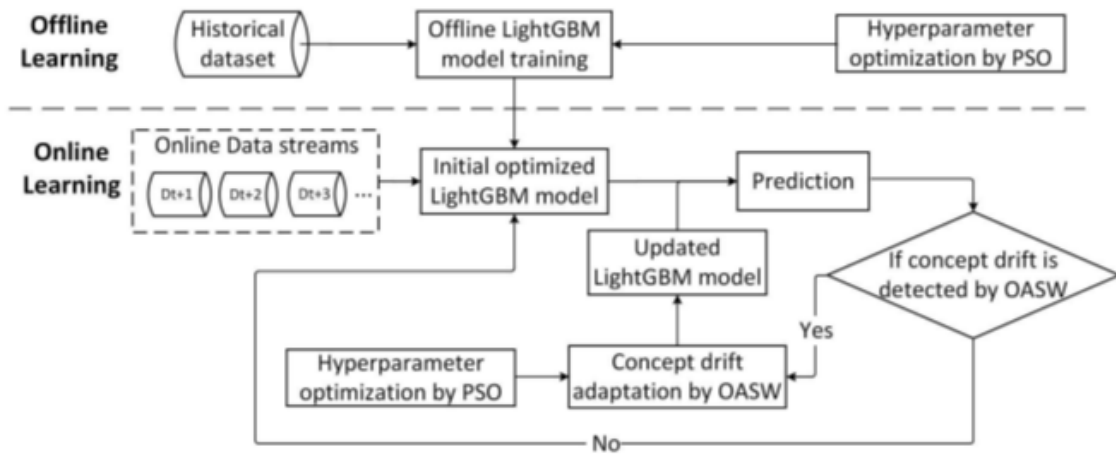


Figure 2.1: The framework of the proposed drift adaptive IoT data analytics model

Another major step is the introduction of OASW, which in addition to drift detection also supports adaptation using a combination of sliding and adaptive windowing. This two-pronged strategy makes it easier to detect the drift and quickly adapt the model for any type of drift. The sliding windows make sure that the model checks for shifts in real time while the adaptive windows enable return to source for more data and retraining of the model.

However, although the provided framework looks rather as a lightweight solution for the IoT systems, it can still have some shortcomings. First, a window-based approach, though

accurate, may not detect the evolution of multi-modal types that happen across out layers of data. For example, IoT systems produce data streams that include features that are often strongly dependent on each other, while the sliding window may only use simple statistical tests to detect changes. Furthermore, the approach of relying on performance degradation to initiate the drift adaptation in OASW may take turns before it degrades and triggers the time for retraining.

Another disadvantage is the applicability of the binary classification for anomaly detection, which hinders its application in more complex scenarios based on IoT, when it is necessary to solve multi-class or regression problems. While binary classification is important in many IoT applications such as predicting when a machine needs maintenance or when a building needs more energy, it does not allow for the type of analysis needed for a more complex application such as prediction maintenance or energy optimization. Last but not least, the paper lacks an analysis of how continuous learning is performed in the context of the added frequency with which IoT data is updated, and models get deployed without requiring being retrained occasionally.

2.1.1.3 Relationship to the proposed research work

The proposed framework provides several key insights on how real-time concept drift detection and adaptation are important and relevant to real-time IoT systems such as supply chain management and e-commerce which are currently under development. LightGBM proves to be a very efficient model to use especially in resource-constrained environments where only computational power and memory are available. When large amounts of streaming data are produced in supply chain systems, LightGBM, being an adaptive model optimized for high dimensional data, can be employed to improve prediction accuracy with minimal resource usage.

While the framework has strengths, it has also left gaps that can be closed with more sophisticated adaptation mechanisms. One example is expanding OASW's drift detection, which is window-based and effective for some types of drift, to work with more complex data relationships. Many real-world systems such as the supply chain exhibit features that are interrelated, and it is possible to have stars without a loss of performance for a long time. A more holistic approach to drift detection would be to integrate more advanced techniques that can perceive feature correlations or temporal dependency changes.

The second gap in the paper is its binary focus, and specifically in anomaly detection tasks. Most of the existing IoT use cases, and extension of this to supply chain and e-commerce, need to process multi-class classification or regression tasks where prediction of better grain categories is essential. To address this limitation, the model is extended to handle multiple classes, thereby making this approach applicable to a wider class of real-time problems.

Finally, although the paper tackles source drift within resource-restricted problems, one could also exploit the opportunity for continuous learning without retraining. It can be very important when data changes quickly and constantly in a domain to be able to learn incrementally, without stopping the application from updating the model. Future work in this area could explore integrating more seamless adaptation techniques that allow the system to continuously learn and adapt without frequent retraining cycles, ensuring greater resilience in dynamic environments.

2.1.2 Disposition-Based Concept Drift Detection and Adaptation in Data Stream

2.1.2.1 Summary of the research item

The paper (2) introduces the Disposition-Based Concept Drift Detection Method (DBDDM) as a novel concept to enhance accuracy in detecting drift and the significant problem of false positive genuine drift. In data streams, concept drift means that in a stream the distribution of data has changed resulting in wrong or outdated results from trained learning models. The previous drift detection techniques are problematic in their ability to distinguish between real drifts from noise and outliers. In such a case this often leads to upgrading the models when it's not necessary, which in turn compromises the quality of the models.

DBDDM presented a two-window system that has an anchored window of old records and a current window for newly received data. The method uses a statistical randomization test wherein the significance of changes between the two windows is determined. DBDDM can then compare the distribution of the data in these windows to decide whether a significant drift has occurred or not. If the drift is large, the model is calibrated; otherwise, the system does not recognize the drift at all. By experiments with different synthesized and real datasets, the paper proves that DBDDM can boost classifier accuracy by up to 28% with classifiers like Naive Bayes and Hoeffding Trees.

2.1.2.2 Critical analysis of the research item

One of the biggest strengths of DBDDM is that the method proposed by the authors aims at the reduction of false positives regarding drift detection, which in turn contributes to more stabilization of the models applied in the context of a changing environment. The two-window comparison system also offers a more refined notion of changes in data, which enables model updates only when a change a more appropriate analysis of data is evident. By excluding noise and random oscillations, the holdout set prevents the model from degrading in its performance with time and keeps its qualities valid.

However, a few factors make this approach desirable but also has some weaknesses. The fixed-size two-window mechanism may not be very effective for a fast-changing environment where the dynamics of concept drift may not be similar. Due to the discreteness and stationarity of the comparison, the chosen window size might either miss some drifts or, vice versa, detect them for too long because of the constantly changing data streams in different media and situations. The use of statistical testing also increases the computational burden, which is measurably damaging in certain high-speed streaming applications that require real-time execution. Furthermore, though the proposed method appears promising for binary or multi-class classification problems, its performance in other large-scale and complex problems, for instance with high dimensionality data or nonlinear interactions between its features, is yet to be better understood.

The other widely overlooked fact about DBDDM is its lack of specificity towards certain types of classification problems. The practical necessities, in particular those of supply chain management and e-buying operations, commonly face repeated data patterns resulting from shifts in demand or supplier performance. Still, the methodology fails to describe how we can handle these periodic drifts that make our models accurate in the long run.

2.1.2.3 Relationship to the proposed research work

This research uncovers important facets that have been overlooked in previous methods regarding the ability to flexibly detect drift. One drawback of DBDDM is that it is static and uses two windows that change frequently and may not capture well in rapidly changing environments. A more dynamic mechanism that can change its window size in a bid to adapt to the observed drift rate would even help, especially in real-time controlling where timely detection is important.

This paper exclusively concentrates on binary and multi-class classification and does not consider the impact of data structure where dependency between the features is important to analyze drift. Thus, to overcome this limitation, the proposed approach introduces artificial intelligence algorithms enabling the recognition of various relationships in data, including complex ones. This enhancement would also make sure that a model can not only be precise but also resistant to the complexity of real-world data.

A crucial shortcoming that is also observed in DBDDM is that the method does not consider multiple concept drift occurrences. In such a setting as supply chain management, where such patterns tend to repeat themselves, the development of such a system that will be able to identify such patterns when they are likely to repeat will in turn enhance the predictive capacity of such a system. The integration of the proposed history analysis model and the proactive adaptation mechanisms will result in an optimal method for the efficient detection of concept drift and subsequent rectification.

2.1.3 Learning in the Presence of Concept Recurrence in Data Stream Clustering

2.1.3.1 Summary of the research item

(3) aims at responding to an important concern related to concept recurrence in data stream clustering namely, how clusters that have been learned by the system can recur in consecutive data streams. It is most relevant to several real-life situations that are evident from the ability of patterns to be repetitive as influenced by events and occurrences in the external environment like consumer behavior analysis and trends in operations of the financial markets to mention but a few. The authors present an algorithm that is intended to recognize these similar ideas more effectively and more rapidly through employing models that have been learned and archived before.

As shown in 2.2, the algorithm is active in two stages that are quite distinct in their processes. The first one is an online phase which is concerned with calculating summary for the accumulating data points and the second one is an offline phase where macro-clusters are produced from the summarized data points. In the online phase, necessary statistics as linear combinations and sums of squares are obtained and saved for later use. Thus, new data entering the system will be classified and integrated into the proper models that represent previously learned notions. The algorithm proves to increase efficiency and decrease the degree of computational complexity achieved by retrieving these models as opposed to training new ones from the ground up. To support their claims, the authors employ a synthetic dataset and two real-world datasets to show not only the capability of their approach in identifying recurring concepts but also the lesser space required as well as the fewer computations needed compared to other approaches.

This idea is based on the belief that data streams are usually non-stationary; in other words, the distribution of the data might change with time. This characteristic makes traditional clustering methods unsuitable since these algorithms do not take into consideration the repetition of previously learned concepts. To this end, focusing on this gap, the paper stresses the need to design algorithms to track and adapt prior knowledge to enhance real-time performance.

2.1.3.2 Critical analysis of the research item

This work may be considered to be a great contribution to the existing research in the area of data stream analysis where the aspect of recurring concept drift is typically overlooked. The algorithm in question also allows for reusing models previously learned to achieve model adaptation while easing the computational load generally inherent to continual learning. This aspect is helpful in a situation of unsupervised learning whereby drift

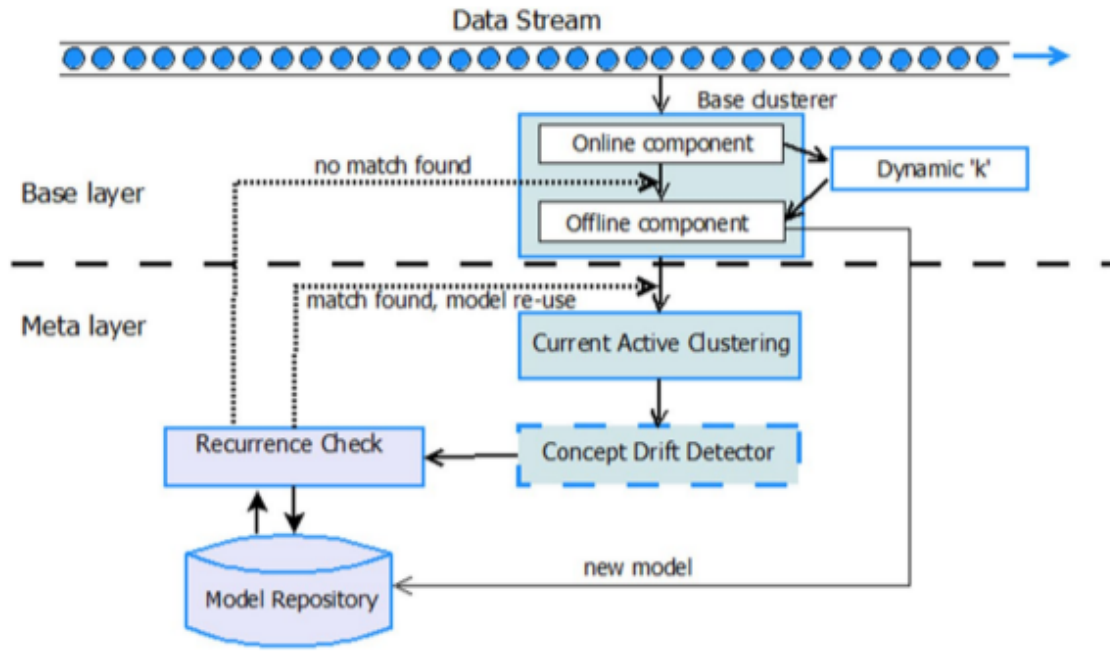


Figure 2.2: Proposed Learning System

detection methods would be difficult due to a lack of labeled data. The use of efficiency in the deployment process means that models can respond accordingly to changes in the distribution of data.

The reliance on clustering techniques, such as CluStream, may confine the algorithm's applicability to situations where data can be grouped into clusters. While working with high-dimensional data or, in other words, data with many correlated features, it may be difficult for the algorithm to identify and react to concept recurrence properly. For example, if data seems bi-modal or is somehow intermixed or there are overall proximity points in many cases the traditional clustering algorithms may not be able to capture micro-decimal variations in data distribution which is crucial in drift detection.

The model retrieval process also assumes that the stored models will be useful for long periods, something that may not be the case where data is rapidly changing with time. In such cases, if the models that are in use are old, the probability of being off the actual results is high which in extension plays a major toll on the efficiency of the whole system. In addition, though the paper focuses on the aspects of unsupervised learning, the research does not devote enough discussion to the issue of how one can combine the approach with the application of supervised learning components. In many applications it is common to have some level of semi-supervised learning, many problem domains involve a combination of labeled and unlabeled data, such as in supply chain management.

2.1.3.3 Relationship to the proposed research work

This paper specifically explores concept recurrence for real-time data stream applications. The focus on clustering and reuse brings valuable insights but also reveals a critical shortcoming in the ability of the algorithm to adapt quickly to changing environments. Rather, we need a more flexible system that not only retrieves past models but also assesses their current relevance given current incoming data characteristics. If models are to remain applicable as data streams continue to evolve, dynamic model updating mechanisms must be included.

There are also limitations in using only clustering techniques to detect drift because improved results can be achieved by supplementing clustering techniques with techniques that involve some level of supervision. This would be universally useful in environments with complex relationships between data points. Closing some important gaps in this existing literature would entail the development of a system that can adaptively switch between clustering for initial data insights and supervised learning for predicting more precisely.

The paper suffers from a lack of predictive capabilities regarding recurring concepts that require future exploration. If a framework can be determined that can tell us when a concept is likely to recur, and also say what conditions contribute to its reappearance, this would enhance the model's overall responsiveness and effectiveness. While this would have also increased adaptation speed, the system would have been empowered to actively pre-condition on known patterns of drift.

Finally, mechanisms to capture contextual information of data streams could be combined with the understanding of concept recurrence to significantly enhance our understanding of concept recurrence. It would mean embedding context-aware features into the model to tell us not only when concepts occur, but under what circumstances will they do so. This capability would enable a more sophisticated and diagnostic view of concept drift, which could then be exploited to achieve better-informed decision-making and better model performance.

2.1.4 Nacre: Proactive Recurrent Concept Drift Detection in Data Streams

2.1.4.1 Summary of the research item

(4) Provides an introduction to Nacre, a framework for identifying concept drift in data streams. Concept drift is a phenomenon that describes changes in data distributions so that the performance degrades when the changes are not detected on time. Nacre incorporates a drift coordinator component that increases the activity of drift detectors and provides a

communication channel between a periodic drift classifier and a sequence predictor. Figure 2.3 shows how the NACRE works visually. This approach enables the identification of subsequent drift points and makes classification more accurate by making adjustments where necessary. Evaluation of Nacre on real (Table 2.5) and synthetic datasets (Table 2.4) shows that the proposed method has higher cumulative accuracy and fewer false positives than previous approaches. Further work involves adding the meta-information and the possibility of automatic adaptation depending on the data stream to improve the results.

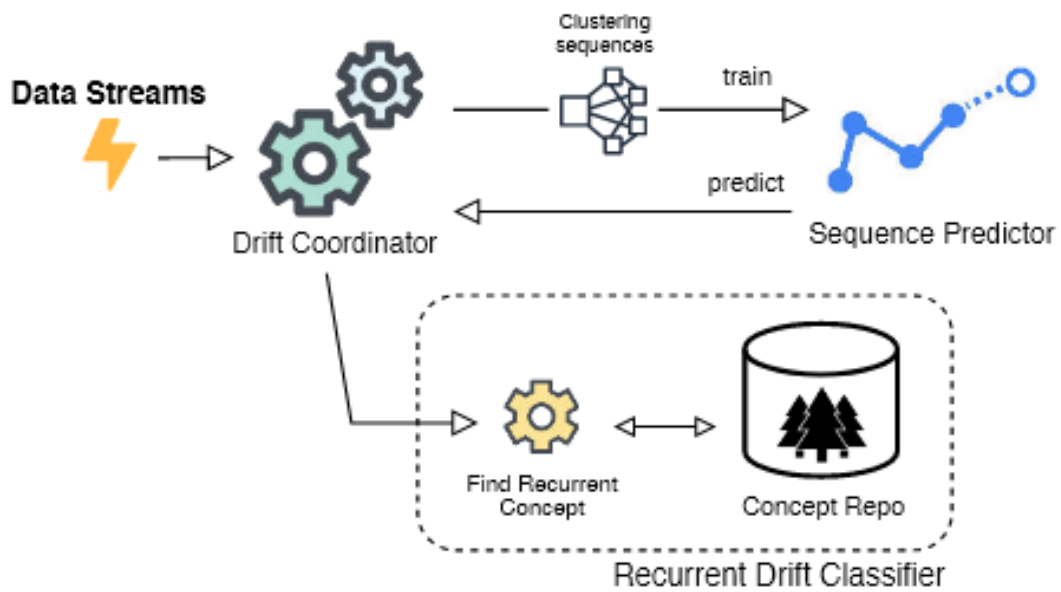


Figure 2.3: NACRE Framework Overview

Dataset	Classifier	Acc. (%)	Kappa (%)	Acc. GPD (%)	Cum. Acc. Gain (%)	Runtime (min)	#Trees
A-P3-A	ARF	72.42 $\pm$ 0.31	17.54 $\pm$ 1.25	-	-	3.50 $\pm$ 0.07	-
	PEARL	80.60 $\pm$ 0.65	40.01 $\pm$ 1.64	8.40 $\pm$ 0.51	4909.88 $\pm$ 301.58	7.07 $\pm$ 0.28	8084 $\pm$ 319
	Nacre	82.12 $\pm$ 0.56	44.45 $\pm$ 1.85	9.91 $\pm$ 0.56	5820.19 $\pm$ 329.00	22.44 $\pm$ 0.69	7417 $\pm$ 220
A-P3-G	ARF	71.97 $\pm$ 0.51	15.10 $\pm$ 0.97	-	-	3.93 $\pm$ 0.11	-
	PEARL	78.24 $\pm$ 0.59	32.68 $\pm$ 2.52	6.61 $\pm$ 0.90	3757.44 $\pm$ 511.50	8.15 $\pm$ 0.11	5297 $\pm$ 226
	Nacre	79.21 $\pm$ 0.37	35.48 $\pm$ 1.78	7.59 $\pm$ 0.49	4340.10 $\pm$ 327.33	20.66 $\pm$ 0.86	5356 $\pm$ 252
A-P10-A	ARF	73.94 $\pm$ 0.47	17.59 $\pm$ 1.43	-	-	3.97 $\pm$ 0.06	-
	PEARL	80.96 $\pm$ 0.63	37.30 $\pm$ 2.38	7.46 $\pm$ 0.84	4210.93 $\pm$ 523.36	7.59 $\pm$ 0.16	4707 $\pm$ 201
	Nacre	83.10 $\pm$ 0.72	43.94 $\pm$ 3.16	9.51 $\pm$ 0.85	5494.04 $\pm$ 535.16	17.74 $\pm$ 0.49	4320 $\pm$ 129
A-P10-G	ARF	72.96 $\pm$ 0.47	14.57 $\pm$ 1.45	-	-	4.22 $\pm$ 0.13	-
	PEARL	76.60 $\pm$ 0.80	24.70 $\pm$ 2.92	4.00 $\pm$ 1.08	2184.63 $\pm$ 581.16	8.15 $\pm$ 0.19	4152 $\pm$ 232
	Nacre	78.46 $\pm$ 0.85	30.44 $\pm$ 2.77	5.77 $\pm$ 1.02	3298.49 $\pm$ 488.12	17.44 $\pm$ 0.71	4219 $\pm$ 246
A-U-A	ARF	73.92 $\pm$ 0.55	16.30 $\pm$ 1.77	-	-	3.95 $\pm$ 0.08	-
	PEARL	81.01 $\pm$ 1.35	36.47 $\pm$ 4.47	7.26 $\pm$ 1.20	4251.92 $\pm$ 719.28	7.57 $\pm$ 0.35	4688 $\pm$ 428
	Nacre	82.92 $\pm$ 1.22	42.40 $\pm$ 4.46	9.06 $\pm$ 1.44	5398.48 $\pm$ 876.71	17.17 $\pm$ 0.59	4350 $\pm$ 321
A-U-G	ARF	72.68 $\pm$ 0.42	15.54 $\pm$ 1.81	-	-	4.27 $\pm$ 0.11	-
	PEARL	76.48 $\pm$ 0.75	25.90 $\pm$ 3.20	3.78 $\pm$ 0.71	2279.30 $\pm$ 413.72	8.27 $\pm$ 0.23	4163 $\pm$ 222
	Nacre	78.05 $\pm$ 0.55	30.66 $\pm$ 2.85	5.31 $\pm$ 0.72	3221.78 $\pm$ 395.13	17.50 $\pm$ 0.47	4229 $\pm$ 190
R-P3-A	ARF	56.72 $\pm$ 0.44	7.48 $\pm$ 0.76	-	-	10.57 $\pm$ 0.70	-
	PEARL	56.98 $\pm$ 0.42	9.77 $\pm$ 1.43	0.32 $\pm$ 0.61	156.68 $\pm$ 418.92	13.34 $\pm$ 0.49	268 $\pm$ 31
	Nacre	57.20 $\pm$ 0.36	10.26 $\pm$ 1.40	0.51 $\pm$ 0.60	288.59 $\pm$ 405.44	13.75 $\pm$ 0.42	270 $\pm$ 35
R-P3-G	ARF	57.33 $\pm$ 0.48	8.22 $\pm$ 0.62	-	-	9.91 $\pm$ 0.56	-
	PEARL	58.25 $\pm$ 0.43	12.71 $\pm$ 1.12	0.50 $\pm$ 0.70	549.25 $\pm$ 343.28	13.26 $\pm$ 0.56	250 $\pm$ 32
	Nacre	58.55 $\pm$ 0.45	13.31 $\pm$ 1.04	0.85 $\pm$ 0.63	729.73 $\pm$ 309.48	13.94 $\pm$ 0.33	257 $\pm$ 33
R-P10-A	ARF	58.17 $\pm$ 0.37	6.82 $\pm$ 0.91	-	-	9.81 $\pm$ 0.71	-
	PEARL	59.01 $\pm$ 0.82	13.18 $\pm$ 2.06	1.39 $\pm$ 0.61	504.75 $\pm$ 507.10	14.91 $\pm$ 0.48	313 $\pm$ 54
	Nacre	59.56 $\pm$ 0.79	14.14 $\pm$ 1.98	1.78 $\pm$ 0.50	831.07 $\pm$ 447.16	15.81 $\pm$ 0.52	319 $\pm$ 58
R-P10-G	ARF	58.22 $\pm$ 0.33	6.50 $\pm$ 1.01	-	-	9.37 $\pm$ 0.73	-
	PEARL	59.80 $\pm$ 1.20	13.58 $\pm$ 3.08	1.46 $\pm$ 1.12	949.39 $\pm$ 679.17	14.90 $\pm$ 0.31	311 $\pm$ 57
	Nacre	60.00 $\pm$ 1.11	14.03 $\pm$ 2.88	1.63 $\pm$ 1.12	1065.26 $\pm$ 652.40	15.74 $\pm$ 0.36	314 $\pm$ 59
R-U-A	ARF	58.03 $\pm$ 0.40	7.25 $\pm$ 0.74	-	-	10.07 $\pm$ 0.71	-
	PEARL	59.14 $\pm$ 0.53	13.42 $\pm$ 1.68	1.16 $\pm$ 0.86	662.61 $\pm$ 496.38	14.88 $\pm$ 0.43	314 $\pm$ 37
	Nacre	59.53 $\pm$ 0.54	14.24 $\pm$ 1.39	1.55 $\pm$ 0.70	898.82 $\pm$ 411.23	15.78 $\pm$ 0.32	318 $\pm$ 43
R-U-G	ARF	58.13 $\pm$ 0.27	7.49 $\pm$ 0.84	-	-	9.90 $\pm$ 1.05	-
	PEARL	59.09 $\pm$ 0.49	13.29 $\pm$ 1.51	0.89 $\pm$ 0.57	580.52 $\pm$ 337.08	14.96 $\pm$ 0.69	269 $\pm$ 42
	Nacre	59.77 $\pm$ 0.40	14.64 $\pm$ 1.11	1.60 $\pm$ 0.42	986.37 $\pm$ 262.25	15.75 $\pm$ 0.46	280 $\pm$ 44

Figure 2.4: Synthetic Dataset Performance

Dataset (#Samples)	Classifier	Acc (%)	Kappa (%)	Cum. Acc. Gain (%)	Runtime (min)	#Trees
Insect incremental-reoccurring balanced (79,986)	ARF	47.88 $\pm$ 19.03	4.92 $\pm$ 13.69	-	0.33	-
	PEARL	56.00 $\pm$ 17.58	27.17 $\pm$ 21.58	642	0.49	280
	NACRE	57.36 $\pm$ 17.86	29.66 $\pm$ 22.27	749	0.63	283
Insect incremental-reoccurring imbalanced (452,044)	ARF	46.11 $\pm$ 10.03	10.19 $\pm$ 14.63	-	2.68	-
	PEARL	68.46 $\pm$ 11.63	52.14 $\pm$ 20.29	10101	6.59	384
	NACRE	69.00 $\pm$ 11.01	53.05 $\pm$ 19.40	10346	8.82	362
Airlines (539,383)	ARF	65.03 $\pm$ 6.18	12.66 $\pm$ 11.48	-	3.45	-
	PEARL	65.73 $\pm$ 5.67	17.20 $\pm$ 12.05	375	6.31	1088
	NACRE	65.75 $\pm$ 5.74	17.05 $\pm$ 12.22	387	4.57	1160
Covtype (581,012)	ARF	66.37 $\pm$ 13.53	19.18 $\pm$ 23.07	-	4.37	-
	PEARL	70.75 $\pm$ 12.71	28.98 $\pm$ 27.70	2543	7.99	10919
	NACRE	71.65 $\pm$ 12.29	31.10 $\pm$ 27.68	3065	11.89	10921
Sensor (2,219,803)	ARF	20.17 $\pm$ 9.18	17.28 $\pm$ 9.81	-	11.73	-
	PEARL	33.94 $\pm$ 15.66	31.75 $\pm$ 16.41	30554	24.07	555
	NACRE	35.34 $\pm$ 16.97	33.22 $\pm$ 17.70	33656	33.15	499

Figure 2.5: Real-world Dataset Performance

2.1.4.2 Critical analysis of the research item

The NACRE framework is a substantial improvement over the existing state of the art for conceptual drift detection in non-stationary domains. Its greatest asset is the online update, whereby the system can update itself without a major shift from the model. Depending on the changes in the concepts, this feature allows avoiding sharp transitions between the concepts and, therefore, mitigating the performance degradation during drifts. The framework uses a recurrent drift classifier and apply the PEARL method that combines different types of trees, including the foreground tree, the background tree, and the candidate tree to observe the recurrent drifts and increase the accuracy of the prediction.

Furthermore, the incorporation of a drift sequence predictor enables future drifts to be forecasted depending on intervals in the past, which makes the system accurate in the long run. The evaluation of the parameters of the framework shows that the values are stable, which implies that there will be little effect on the performance in stable data stream so making deployment and tuning easier. However, there are some weaknesses associated with the NACRE framework as well. This is because its implementation involves several components; namely the drift classifier, predictor, and coordinator.

Moreover, the accuracy of the drift sequence predictor is highly dependent on the past drift data; if the information available is inadequate or not very relevant, then the predictions developed will not be very sound. The framework may also be prone to overfitting especially in environment with high fluctuations of data streams since specific models are difficult to adapt to new situations. Therefore, one concludes that, although the NACRE framework offers a potential solution for the proactive detection of drift, the directions for further improvement have been also identified. Integrating meta-information into the drift coordinator and making the coordinator capable of tuning itself to the characteristics of different data streams are the next logical steps to improve the drift coordinator. These improvements will not only fill the existing gaps but also will bring the framework to the optimum level of development. Thus, the focus on these improvements presupposes the further enhancement and flexibility of the NACRE framework as the tool for combating concept drift in a number of applications.

2.1.4.3 Relationship to the proposed research work

The Nacre framework has further improved in handling concept drift, and more so with the drift coordinator component. However, there exist crucial areas that have not yet been enhanced fully, such as the integration of meta-information and the possibility of automatic changing of the drift coordinator.

- **Meta Information Incorporation:** The addition of meta-information into the drift coordinator would be very valuable for the current Nacre framework. This alter-

ation might help enhance the outcomes of drift estimates by adding more context about the data—such as history or outside conditions. Richer data context allows the drift coordinator to make better decisions, thus better handling of concept drifts and minimizing the loss of performance during the detection phase.

- **Automatic Adaptation of the Drift Coordinator:** While Nacre has a drift coordinator, it does not utilize all of the potential for automatic adjustment of the employed methods based on the data stream properties. This proposed research aims at establishing a dynamic drift coordinator where the settings can be adjusted based on data conditions. This flexibility would improve on the framework’s performance by providing faster and more appropriate responses to the changes in data.

Therefore, with these two improvements in mind, this research seeks to address existing gaps within the Nacre framework. These improvements will contribute to the creation of a stronger and more accurate system for real-time concept drift detection, which will be more applicable in practice such as supply chain management and e-commerce.

2.1.5 Recurrent Adaptive Classifier Ensemble for Handling Recurring Concept Drifts

2.1.5.1 Summary of the research item

The work of Museba et al. (2021) (5) introduces the Recurrent Adaptive Classifier Ensemble (RACE) which is a new approach to dealing with recurring concept drifts in real-world data streams. RACE addresses the concept drift in terms of data distribution since the data distribution may change over time, and RACE is directed to recurring concepts, which can appear in cycles.

RACE employs Support Vector Machines (SVMs) as base models, with an archive of previously learned models of diverse types to quickly transition based on new data, while retaining the learned knowledge. In the case of the arrival of a new data chunk, RACE is used to assess whether this data chunk follows the distribution of the previous data. If there is a change, the current support vector is modified, and a new support vector is procured.

Unlike conventional methods, where mean predictions are obtained from stored models, RACE fine tunes these models to the data available today thus providing better accuracy. The algorithm targets diversification of models and uses Yule’s Q Statistic in order to mitigate errors in the ensemble. By taking this approach on account of diversity, RACE is able to learn more efficiently and with better precision the recurring concepts with low computational cost. Quantitative experiments on synthetic and real applications demon-

strate that RACE yields a considerably better performance than other ensemble classifiers in terms of handling multiple occurrences of concept drift.

2.1.5.2 Critical analysis of the research item

The paper presents a new and more flexible algorithm called the Recurrent Adaptive Classifier Ensemble (RACE) for handling repeated notions in data streams. RACE distinguishes itself by storing previously learned, yet very diverse models which are then adapted to adapt to the new data chunks. To assess the performance of RACE in repeated concept scenarios the authors performed two empirical studies in streaming environments. In the first experiment, they built the heterogenous classifier set and, applying knowledge transfer, chose the most appropriate models that reflect the current concept in the latest data.

A drift detection mechanism was included to determine if a concept drift has happened. The findings revealed that the proposed method of using knowledge transfer and drift detection improves the performance of the algorithm by a huge percentage when working with nonstationary time series data. The second experiment considered the behavior of RACE when the knowledge transfer was substituted by the Hidden Markov Model for the forecast of the further drifts, but this method was ineffective for the processing of the nonstationary time series data streams. In general, this research shows that RACE provides new opportunities for studying how new concepts can be learned with low cost and time and with minimal computational load in nonstationary contexts, which are poorly addressed in the literature. However, as a newly developed algorithm, the RACE algorithm has some drawbacks. Below tables provide an insight to memory consumption and CPU processing times of RACE with other recurrent drift models.

Dataset	RACE	CALMID	DUE	SOFEnsemble	AWDOB
Airlines	36.25 (1)	42.39 (2)	44.29 (3)	57.31 (4)	61.23 (5)
Coverttype	43.17 (2)	28.34 (1)	63.46 (5)	59.63 (4)	51.42 (3)
Poker Hand	66.37 (1)	71.26 (3)	69.47 (2)	79.45 (5)	77.36 (4)
KDD99	83.43 (4)	76.39 (2)	78.36 (3)	73.43 (1)	88.34 (5)
Sensor Data	59.34 (1)	87.63 (5)	67.34 (3)	79.23 (4)	62.41 (2)
Average rank	1.8	2.6	3.2	3.6	3.8

Figure 2.6: RACE Memory Consumption

Dataset	RACE	CALMID	DUE	SOFEnsemble	AWDOB
Airlines	13.49 (1)	15.43 (2)	29.43 (3)	54.03 (5)	35.38 (4)
Coverttype	17.36 (2)	36.48 (3)	33.42 (2)	53.48 (4)	65.43 (5)
Poker Hand	43.56 (1)	78.45 (4)	83.37 (5)	73.28 (3)	56.32 (2)
KDD99	81.43 (3)	78.32 (2)	76.43 (1)	88.46 (5)	83.52 (4)
Sensor Data	85.46 (2)	81.34 (1)	91.23 (3)	99.38 (5)	97.41 (4)
Average rank	1.8	2.4	2.8	4.4	3.8

Figure 2.7: RACE CPU Processing Time

The major difficulty can be its computational complexity, which may demand a significant amount of memory to store the variety of classifiers and to control the concept transfers. Furthermore, as the size of the ensemble increases, convergence to recurring concepts may take longer due to more time being required to transfer concepts and this may be disadvantageous in real time classification. However, this paper provides a basis for future work and the exploration of other techniques to address the issue of repeated ideas within nonstationary time series data with the purpose of comparing it with the new RACE algorithm.

2.1.5.3 Relationship to the proposed research work

The current research extends on enhancing the NACRE framework, as it includes several shortcomings, which have been identified in the prior models such as RACE. Even though RACE shows a good starting point for addressing recurring concepts in nonstationary data, the algorithm is rather intricate, requires much memory, and is computationally heavy, which renders it infeasible for many realistic tasks, particularly in constrained environments. Similarly, NACRE also suffers from scalability issues as well as adaptability and efficiency when it deals with concept drift in large scale data streams.

The proposed research deals with enhancing the NACRE model by including meta-information in order to increase the accuracy of drift detection and address the issue of data streams heterogeneity. Furthermore, a lighter version of the framework will be created to reduce the computational cost which will then be appropriate for particular application areas such as supply chain and e-commerce datasets. This research will target at revisiting the drift detection and model adaptation process, which have been made inefficient in both NACRE and other comparable models like RACE; the aim here will be to make them efficient and practical for use in specific domains.

Chapter 3

Proposed Approach

3.1 Workflow of the Approach

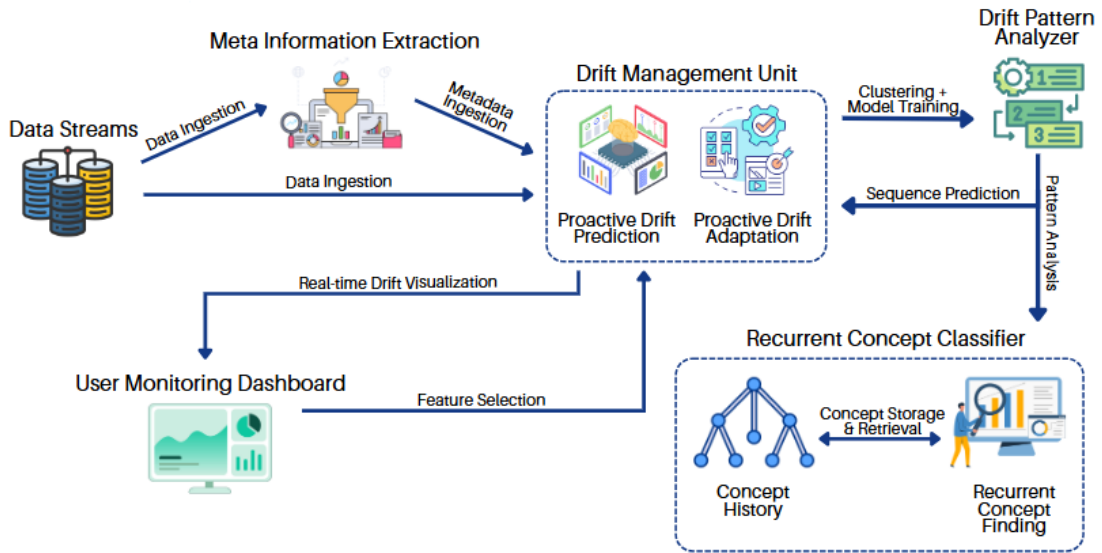


Figure 3.1: Proposed workflow for DriftGuard

The proposed framework 3.1 for detecting and managing recurring concept drift in data streams is structured around a well-defined architecture that consists of several key components: Meta Data, Data Stream, Drift Management Unit, Recycling Drift Pattern Analyzer, and Recurrent Drift Classifier. Initially, the Data Stream is the main source of real-time data, autonomously feeding operational metrics as a result of the execution of supply chain activities, customer interactions, and inventory levels. Essentially, the Meta Data component contextualizes and enriches this data with data types, timestamps, and relationship data regarding different data points. Hence this information; the context helps

in detecting changes in data or concept drift. The Monitoring Dashboard is viewed as a user-friendly interface to view and track the key performance indicators(KPIs) and to track model performance and drift detection in real-time. Stakeholders can interact with the system, view data trends, and spot possible drift happening quickly with this dashboard. The Drift Management Unit, at the heart of the architecture, detects concept drift using different algorithms, which is at the heart of architecture. It compares incoming data to historic patterns and when drift is suspected it triggers alerts. Next, detected drifts are analyzed by the Drift Pattern Analyzer to identify the nature of drift, whether recurrent, gradual, or quick, and characterize the pattern behind drift for appropriate responses. Finally, the Recurrent Drift Classifier identifies recurring drifts classifies the discovered recurring drifts, and performs necessary adaptations to the models to keep the system responsive and accurate with time.

3.2 Key features of the proposed approach

Several key features are built into DriftGuard to improve its ability to detect and handle concept drift. The biggest feature is its anticipatory drift detection mechanism that predicts possible drifts based on historical data patterns in the stream. DriftGuard uses a Temporal Pattern Graph to analyze past drift occurrences to find correlations and facilitate proactive adaptation before drift negatively impacts predictive accuracy. This capability allows the system to anticipate drift instead of just reacting to them, which improves stability in overall performance.

Also, handling multi-modal drift is another crucial feature that is often present in such complex data spaces. It allows DriftGuard to maintain and adapt multiple models in parallel in response to a combination of simultaneous changes in various subsets of monitored data. This becomes critical when many factors affect various elements of performance, specifically in scenarios where influence on demand and logistical constraints, while the same, affect different areas differently.

Existing frameworks only replace models when they detect drift, while DriftGuard allows incremental model refinement. Hence, rather than discarding the models altogether, the framework can continue to revise existing models according to new incoming data. This hybrid approach allows the system to learn and adapt while maintaining its inherent performance levels. DriftGuard also uses an adaptive forgetting mechanism to manage the size of the Tree Transition Graph. With this mechanism, we can retain critical transitions associated with recurring drifts while removing less significant edges, preserving valuable historical information that can inform future adaptations.

Algorithm 1 Initialize DriftGuard Model Parameters (Modified PEARL)

```

1: Initialize DriftGuard:
2:   foreground_trees  $\leftarrow$  ARF_foreground_trees()
3:   background_trees  $\leftarrow$  ARF_background_trees()
4:   online_tree_repository  $\leftarrow$  {}
5:   candidate_trees  $\leftarrow$  {}
6:   Tree_Transition_Graph  $\leftarrow$  {}
7:   Temporal_Pattern_Graph  $\leftarrow$  initialize_temporal_graph()
8:   Multi_Modal_Subsets  $\leftarrow$  initialize_modal_subsets()
9:   Forgetting_Mechanism  $\leftarrow$  Adaptive_Forgetting_Mechanism()
10:  Lossy_Counting_Window  $\leftarrow$   $\frac{1}{\epsilon}$ 

```

Algorithm 2 Main Function for Data Stream Processing (DriftGuard)

```

1: for each new_data_instance in data_stream do
2:   predictions  $\leftarrow$  []
3:   for each tree in foreground_trees do
4:     prediction  $\leftarrow$  tree.predict(new_data_instance)
5:     Append prediction to predictions
6:   end for
7:   for each tree in foreground_trees do
8:     if tree.detect_warning(new_data_instance) then
9:       background_tree  $\leftarrow$  background_trees[tree]
10:      if tree.detect_drift(new_data_instance) then
11:        replace_drifted_tree(tree, background_tree, candidate_trees)
12:      end if
13:      (New) anticipate_drift(new_data_instance, Temporal_Pattern_Graph)
14:    end if
15:  end for
16: end for

```

Algorithm 3 Anticipate Drift and Model Adaptation (New in DriftGuard)

```

1: function ANTICIPATE_DRIFT(data, Temporal_Pattern_Graph)
2:   predicted_drift  $\leftarrow$  predict_drift_from_temporal_graph(Temporal_Pattern_Graph, data)
3:   if drift_is_likely(predicted_drift) then
4:     apply_proactive_adaptation()
5:   end if
6: end function

```

Algorithm 4 Drift Replacement Mechanism with Model Refinement (Improved)

```

1: function REPLACE_DRIFTED_TREE(tree, background_tree, candidate_trees)
2:   if candidate_trees exists then
3:     selected_tree  $\leftarrow$  select_best_candidate(candidate_trees)
4:   else
5:     selected_tree  $\leftarrow$  background_tree
6:   end if
7:   foreground_trees.replace(tree, selected_tree)
8:   if requires_refinement(tree) then
9:     refine_tree(tree) ▷ (New) Incremental model refinement
10:  end if
11:  online_tree_repository.add(tree)
12:  candidate_trees  $\leftarrow$  select_candidate_trees(online_tree_repository, tree)
13: end function

```

Algorithm 5 Multi-Modal Drift Handling (New in DriftGuard)

```

1: function HANDLE_MULTI_MODAL_DRIFT(data, Multi_Modal_Subsets)
2:   for each subset in Multi_Modal_Subsets do
3:     if subset.detect_drift(data) then
4:       adapt_subset_models(subset)
5:     end if
6:   end for
7: end function

```

Algorithm 6 Adaptive Forgetting Mechanism (Improved from Lossy Counting)

```

1: function ADAPTIVE_FORGETTING(Tree_Transition_Graph)
2:   for each edge in Tree_Transition_Graph.edges do
3:     if edge.recurrence_interval < threshold then
4:       retain_edge(edge)
5:     else
6:       remove_edge(edge) ▷ Preserve critical edges for recurring drifts
7:     end if
8:   end for
9: end function

```

3.3 Visualization of the Dashboard

An important element of this research framework is the work with a dashboard, as the visualization tool used for the monitoring and analysis of the drift of concepts in real-time streams. This dashboard allows stakeholders to easily comprehend visualizations of how drift affects the model and more crucially how a repeated pattern in datasets may necessitate a redoing of new models to remain accurate.

3.3.1 Purpose of the Dashboard

The first and foremost purpose of the dashboard is to provide the concept drift of the data streams with periodic patterns that occur during the process. In supply chain management and other such fields where large amounts and complex data are being generated, the process of manually identifying these patterns can be extremely onerous and prone to error due to the sheer scale and intensity of information being received. Metrics calculated by the dashboard combined with the Ex collection make it easy for the users to analyze values and check for recurring shifts that are not visible with the standard approach.

Although such a framework can be used for a variety of use cases within the research, the proposed dashboard is equally versatile with regards to data streams that can be visualised. In the context of this section an example visualization based on the comparison of the sales and shipping performance through several years is provided. This example is one single instance and can be taken to other areas, where operational metrics are involved, be it inventory stock, supplier's delivery hours, or customer demand pattern, making it very flexible for the supply chain and e-commerce environments.

One of the visualizations [3.2](#) created for the dashboard is that showing sales and shipping performances over several years and will allow investigating the impact of a drift of the concept in these indicators. The specific example deals with considering patterns of repeated drift that may be observed in the course of particular months during several years. The data set includes monthly data for the total sales for three years from 2015 to 2017, and average shipping days. The visualization annexed hereto shows that there is a cycle of declines in sales in the second month of each year, February 2015, February 2016, February 2017. This decline in sales points to a consistent shift, and this happens on a yearly basis, during the same time of the year. This recurring behaviour can only imply that push or pull factors characteristic with the supply chain negatively impact on the sales during this particular period. Such tendencies can be, for instance, disconnected with the decrease in the demand during holidays, transportation slowdowns, or other reasons which led to cycle decline. Each of these patterns is vividly displayed on the dashboard so that the user can easily appreciate the importance of these repeat slants.

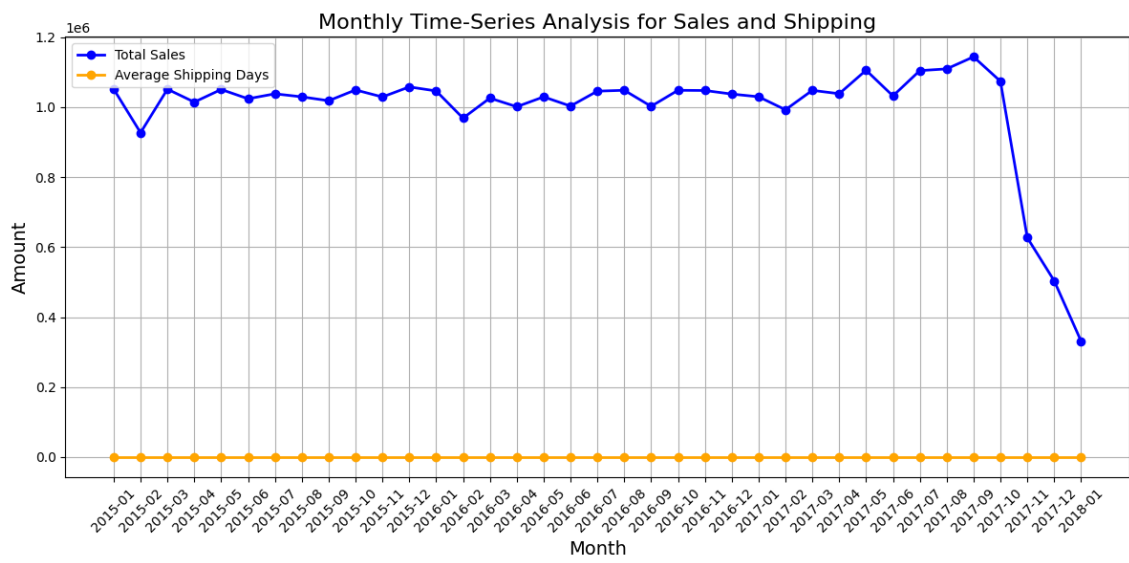


Figure 3.2: Monthly Time-Series analysis for Sales and shipping

Bibliography

- [1] Yang, L. & Shami, A. A lightweight concept drift detection and adaptation framework for IoT data streams. *IEEE Internet Of Things Magazine*. **4**, 96-101 (2021)
- [2] Agrahari, S. & Singh, A. Disposition-based concept drift detection and adaptation in data stream. *Arabian Journal For Science And Engineering*. **47**, 10605-10621 (2022)
- [3] Namitha, K. & Santhosh Kumar, G. Learning in the presence of concept recurrence in data stream clustering. *Journal Of Big Data*. **7**, 75 (2020)
- [4] Wu, O., Koh, Y., Dobbie, G. & Lacombe, T. Nacre: Proactive recurrent concept drift detection in data streams. *2021 International Joint Conference On Neural Networks (IJCNN)*. pp. 1-8 (2021)
- [5] Museba, T., Nelwamondo, F., Ouahada, K. & Akinola, A. Recurrent adaptive classifier ensemble for handling recurring concept drifts. *Applied Computational Intelligence And Soft Computing*. **2021**, 5533777 (2021)